Operating Systems (UE24CS242B)

Building PES-VCS — A Version Control System from Scratch

NAME: Darshana S

SRN: PES1UG24CS913

SECTION: 4J

SCREENSHOTS:

PHASE 1:

1A - ./test_objects output showing all tests passing

```
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B- find .pes/objects -type f showing sharded directory structure

```
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$
```

PHASE 2:

2A- ./test_tree output showing all tests passing

```
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$
```

2B- xxd of a raw tree object (first 20 lines)

```
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$ xxd .pes/objects/f5/902ee73b6aa25436f687067fc4cc0cb481cf416740d3c3c3ed6d9c4de7b6c2| head -20

00000000: 7472 6565 2039 3800 3130 3036 3434 2066  tree 98.100644 f
00000010: 696c 6531 2e74 7874 002c f8d8 3d9e e295  ile1.txt.,..=...
00000020: 43b3 4a87 7274 21fd ecb7 e3f3 a183 d337  C.J.rt!.......7
00000030: 6390 25de 576d b9eb b431 3030 3634 3420  c.%.Wm...100644
00000040: 6669 6c65 322e 7478 7400 e00c 50e1 6a2d  file2.txt...P.j-
00000050: f38f 8d6b f809 e181 ad02 48da 6e67 19f3  ...k......H.ng..
00000060: 5f9f 7e65 d6f6 0619 9f7f                 _.~e......
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$
```

PHASE 3:

3A- pes init → pes add → pes status sequence

```
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$ ./pes add file1.txt file2.txt
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_tree.c
  untracked:  README.md
  untracked:  Makefile
  untracked:  test_objects.c
  untracked:  commit.h
  untracked:  index.h
  untracked:  pes.h
  untracked:  commit.c
  untracked:  pes.c
  untracked:  tree.c
  untracked:  tree.h
  untracked:  test_sequence.sh
  untracked:  object.c
  untracked:  .gitignore
  untracked:  index.c

darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$
```

3B- cat .pes/index showing the text-format index

```
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776517999 6 file1.txt
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776518003 6 file2.txt
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$
```

PHASE 4:

4A- pes log output with three commits

```
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$ ./pes log
commit c5db9a4ce57446ed7ac7be957b85bfd673a4113d8455e52d23528d4ee62af99b
Author: Darshana S <PES1UG24CS913>
Date:   1776519375

    Final commitA

commit 6f4aca41fb6f4165abfc9ad9d69db6a37fce77da3f50934f4bd87b6e5347be48
Author: PES User <pes@localhost>
Date:   1776519016

    Add farewellmmit!

commit 492d7f3fbc5a696b710b9ef0d45616491ced4f3055e9b178e1f01a247507d90d
Author: PES User <pes@localhost>
Date:   1776519016

    Add worldellmmit!

commit 3a4e523b174b01e03577a3ccc3acb9032449cba1f701c38ec4f9d5926d631704
Author: PES User <pes@localhost>
Date:   1776519007

    Initial committial commiQ

darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$
```

4B- find .pes -type f | sort showing object growth

```
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/3a/4e523b174b01e03577a3ccc3acb9032449cba1f701c38ec4f9d5926d631704
.pes/objects/49/2d7f3fbc5a696b710b9ef0d45616491ced4f3055e9b178e1f01a247507d90d
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6f/4aca41fb6f4165abfc9ad9d69db6a37fce77da3f50934f4bd87b6e5347be48
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$
```

4C-cat .pes/refs/heads/main and cat .pes/HEAD

```
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$ cat .pes/refs/heads/main
6f4aca41fb6f4165abfc9ad9d69db6a37fce77da3f50934f4bd87b6e5347be48
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$
```

FINAL SCREENSHOT:

```
darshana@darshana-VirtualBox:~/PES1UG24CS913-pes-vcs$ make test-integration
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 327383d8fec6... Initial commit

Log after first commit:
commit 327383d8fec601b28c789781d09c978ac404a7097e5cf3128bd18c3ff59b58e8
Author: PES User <pes@localhost>
Date:   1776521356

    Initial commit


--- Second Commit ---
Committed: 2104aa2ec826... Update file.txt

--- Third Commit ---
Committed: b58c0dfb85d4... Add farewell

--- Full History ---
commit b58c0dfb85d4cce98d240950ae2abe045e4611ffc02bfbcfe83bbe86a008457e
Author: PES User <pes@localhost>
Date:   1776521356

    Add farewell
```

```
commit 2104aa2ec826c3001cdec2533e4d7e76a1df77653f9863f1bdbf38ba773abae1
Author: PES User <pes@localhost>
Date:   1776521356

    Update file.txt

commit 327383d8fec601b28c789781d09c978ac404a7097e5cf3128bd18c3ff59b58e8
Author: PES User <pes@localhost>
Date:   1776521356

    Initial committial commiQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
b58c0dfb85d4cce98d240950ae2abe045e4611ffc02bfbcfe83bbe86a008457e

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/21/04aa2ec826c3001cdec2533e4d7e76a1df77653f9863f1bdbf38ba773abae1
.pes/objects/32/7383d8fec601b28c789781d09c978ac404a7097e5cf3128bd18c3ff59b58e8
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/b5/8c0dfb85d4cce98d240950ae2abe045e4611ffc02bfbcfe83bbe86a008457e
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Analysis Questions

PHASE 5: Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

ANS. To implement pes checkout <branch>, the file .pes/HEAD must be updated to point to refs/heads/<branch>, and the working directory must be updated to match the tree of the commit referenced by that branch.

This involves:

- Reading the commit object of the target branch
- Extracting its tree object
- Traversing the tree and writing blob contents to the working directory

Files present in the current branch but not in the target must be deleted, files present in both must be overwritten, and new files must be created. Directory structure must also be recreated.

The operation is complex because it requires synchronizing the working directory with the target tree while safely handling file deletions, overwrites, and directory changes.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

ANS. To detect a dirty working directory, for each tracked file in the index, compare its stored metadata (mtime and size) with the actual file on disk. If they differ, the file is considered modified ("dirty").

Next, compare the current index entries with the target branch's tree. If a file differs between branches and is also marked dirty in the working directory, checkout must refuse to avoid overwriting uncommitted changes.

This detection relies only on metadata comparison (mtime and size), avoiding expensive re-hashing.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

ANS. In a detached HEAD state, HEAD points directly to a commit hash instead of a branch reference. Any new commits created in this state are not associated with a branch, so no reference updates to track them.

If the user switches away, these commits become unreachable.

To recover them, the user must create a branch pointing to the commit using:
pes branch <name> <commit-hash>
and then checkout that branch.

PHASE 6: Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

ANS. Garbage collection starts by identifying all reachable objects. Begin from every branch reference in .pes/refs/heads/, then traverse:

- all commits (via parent links)
- their corresponding tree objects
- and recursively all blob objects in those trees

Store all reachable hashes in a hash set for efficient O(1) lookup.

Then scan all objects in .pes/objects/. Any object not present in the reachable set is considered unreachable and can be safely deleted.

For a repository with 100,000 commits and ~10 files per commit, approximately:

- 100,000 commits
- 100,000 trees
- 1,000,000 blobs

→ Around **1.1 million objects** need to be visited.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

ANS. Running garbage collection concurrently with a commit is dangerous due to race conditions.

Example: a commit process creates a tree object but has not yet written the commit object referencing it. If GC runs at this moment, it sees the tree as unreachable and deletes it. The commit then writes a reference to a non-existent tree, corrupting the repository.

Git avoids this using a grace period — recently created objects (e.g., within 2 weeks) are never deleted. Additionally, Git uses mechanisms like temporary references ("keep" files) to protect objects that are in use during operations.